

Examination Timetabling Problem

Tran Nam Hai - 202400103

June 8, 2025

Abstract

The Examination Timetabling Problem (ETP) is a well-known NP-hard combinatorial optimization problem with significant practical relevance in educational institutions. It involves assigning a set of exams to available time slots and rooms while satisfying various hard and soft constraints. This study investigates five solution techniques: a Constraint Programming (CP) model, a Mixed Integer Programming (MIP) model, and three heuristic-based methods. Experimental results demonstrate that the heuristic-based approach consistently outperforms CP and MIP in terms of solution quality and computational efficiency. A comprehensive analysis of the strengths and limitations of each method is presented, along with discussions on possible improvements and future research directions. The findings contribute to the advancement of ETP-solving methodologies and provide actionable insights for practitioners aiming to optimize exam scheduling and related packing problems.

1 Introduction and Objectives

There are N classes $1, 2, \dots, N$ that need to be scheduled for the final exam. Each class must be assigned to a time-slot and a room. There are M rooms $1, 2, \dots, M$ that can be used for scheduling the exam. Each room i has capacity $c(i)$ (number of places of the room). Each day is divided into 4 slots $1, 2, 3, 4$. Each class i has number of students $d(i)$ ($i = 1, \dots, N$). Among N classes, there are K pairs of classes (i, j) in which class i and class j have the same student participating in the exam. It means that these 2 classes cannot be scheduled in the same time-slot. Compute the exam time-table such that the number of days used is minimal.

Data Format

- First line: N and M
- Second lines: $d_1, d_2, \dots, d_N$
- Third lines: $c_1, c_2, \dots, c_M$
- Fourth line: K
- Last K lines: i and j (2 classes having a same student registered, these classes cannot be scheduled in the same slot)

Generated Data

The input size is denoted by $(N \times M)$.

- 15 tests:
 $(5 \times 2), (6 \times 2), (7 \times 3), (8 \times 3), (9 \times 4),$
 $(10 \times 3), (11 \times 4), (12 \times 4), (13 \times 4), (14 \times 4),$
 $(15 \times 4), (16 \times 4), (17 \times 4), (18 \times 5), (19 \times 5)$
- 10 tests:
 $(50 \times 10), (51 \times 10), (52 \times 11), (53 \times 11), (54 \times 11),$
 $(55 \times 11), (56 \times 12), (57 \times 12), (58 \times 12), (59 \times 13)$
- 10 tests:
 $(100 \times 10), (200 \times 10), (300 \times 10), (400 \times 10), (500 \times 10),$
 $(600 \times 11), (700 \times 11), (800 \times 11), (900 \times 10), (1000 \times 10)$
- 3 tests mainly for heuristic testing:
 $(1000 \times 20), (2000 \times 20), (3000 \times 20)$

Each test case is randomly generated as follows:

- Each class has a number of students between 20 and 70.
- Let S be the largest number of students in any class. Each room has a capacity between $S/2$ and $3S/2$.
- The number of pairs of classes that share at least one student is at most $N(N+1)/4$.

2 Modeling the Problem

In the model, let:

- N : the number of classes
- M : the number of rooms
- d_i : the number of students in class i
- c_j : the capacity of room j
- P : the set of conflict pairs (i.e., pairs of classes that share at least one student)

2.1 CP Model

2.1.1 Decision Variables

- $slot_i \in \{1, \dots, N\}$: time-slot assigned to class i
- $room_i \in \{1, \dots, M\}$: room assigned to class i

2.1.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$d_i > c_j \Rightarrow room_i \neq j$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$slot_i \neq slot_j, \forall (i, j) \in P$$

- Only one class per room per time-slot:

$$slot_i = slot_j \Rightarrow room_i \neq room_j$$

2.1.3 Objective Function

Minimize the number of time-slots used:

$$\min \left(\max_{i=1}^N slot_i \right)$$

2.2 MIP Model

2.2.1 Decision Variables

- $x_{i,s,r} \in \{0, 1\}$: binary variable, equals 1 if class i is assigned to time-slot s and room r , otherwise 0
- $D \in [1, N]$: the maximum time-slot index used (i.e., latest slot with an exam scheduled)

2.2.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$x_{i,s,r} = 0 \text{ if } c_j < d_i$$

- Each class is scheduled exactly once:

$$\sum_{s=1}^N \sum_{r=1}^M x_{i,s,r} = 1, \forall i \in \{1, \dots, N\}$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\sum_{valid r} (x_{i,s,r} + x_{j,s,r}) \leq 1, \forall (i, j) \in P, \forall s \in \{1, \dots, S\}$$

- Only one class per room per time-slot:

$$\sum_{i=1}^N x_{i,s,r} \leq 1, \forall s \in \{1, \dots, S\}, \forall r \in \{1, \dots, M\}$$

- Tracking the latest time-slot used:

$$\sum_{s=1}^N \sum_{r=1}^M (s+1) \cdot x_{i,s,r} \leq D, \forall i \in \{1, \dots, N\}$$

2.2.3 Objective Function

Minimize the number of time-slots used:

$$\min D$$

3 Heuristic Methods

We present three approaches using different heuristic algorithms: Simulated Annealing (SA), Beam Search (BS), and Harmony Search (HS).

Custom terms

In this report, the term “cost” refers to the total number of slots required to schedule all exams.

3.1 Simulated Annealing

3.1.1 Overview

Simulated Annealing (SA) is a meta-heuristic inspired by the physical process of annealing in metallurgy. Starting from an initial feasible timetable, SA explores the solution space by probabilistically accepting both improving and, with decreasing probability, worsening moves. This allows the algorithm to escape local minima and gradually “freeze” towards a high quality schedule.

3.1.2 Strategy

Initial Solution

Begin with a greedy assignment of classes to time-slots and rooms, yielding an initial cost.

Temperature Schedule

Set an initial temperature T_0 high enough to allow diverse moves. Decrease T multiplicatively until a minimum temperature T_{min} is reached.

Move Acceptance

At each temperature, randomly select a class and attempt to reassign it. Compute the change in cost Δ . Always accept moves with $\Delta < 0$, otherwise accept with probability $\exp(-\Delta/T)$.

3.1.3 Process

The scheduling procedure begins with a simple greedy initialization. First, we randomly shuffle the list of classes to avoid any bias in ordering. Then, for each class in this shuffled order, we assign it to the earliest possible time slot and room that satisfy capacity and conflict constraints and record this arrangement as our current best solution.

Next, we enter the annealing phase. We start with an initial temperature and continue iterating as long as the temperature remains above a predefined minimum. In each iteration, we select a class and remove it from its current assignment. We then attempt a fixed number of random reassignments for this class by selecting a new slot and room that still respect capacity and conflict requirements. If one of these tentative moves is feasible, we calculate the resulting change in the overall cost. We then decide whether to accept this move based on the Metropolis criterion: if the new cost is lower, we accept it unconditionally and update our best solution if it improves upon the global best; if the cost is higher, we may still accept it with a probability that decreases as the temperature drops. If no acceptable move is found or if a chosen move is rejected, we revert the class to its original slot and room. At the end of each iteration, the temperature is multiplied by a cooling factor, gradually reducing the likelihood of accepting worse moves until the system cools down to the minimum temperature.

Once the annealing loop concludes, we perform a final local refinement to further optimize the timetable. For each class in the current schedule, we check whether moving it to an earlier day can reduce the total number of days used. If such a move strictly improves the schedule, we apply it. This three-stage approach (greedy start, annealing-based exploration, and local tightening) yields a balanced solution that minimizes days while respecting all constraints.

3.1.4 Pseudo-code

The following pseudo-codes describe the implementation of the Simulated Annealing and Local Search algorithms used to solve the ETP. The pseudo-codes outline the steps involved in the scheduling process using these algorithms.

Algorithm 1 The Local Search algorithm

Input: A solution S
Output: Locally optimized solution S
 $\text{improved} \leftarrow \mathbf{true}$
while improved **do**
 $\text{improved} \leftarrow \mathbf{false}$
 for each class c in S **do**
 for each earlier slot s **do**
 for each room r **do**
 if assigning c to (s, r) reduces $\text{cost}(S)$ **then**
 Apply the assignment of c to (s, r)
 $\text{improved} \leftarrow \mathbf{true}$
 break all loops
 end if
 end for
 end for
 end for
end while
return S

Algorithm 2 The Simulated Annealing algorithm

Input: Initial feasible solution S
Output: Optimized solution S^*
 $S^* \leftarrow S, T \leftarrow T_0$
while $T > T_{\min}$ **do**
 for 1 to MaxTrials **do**
 Select a class c and remove its assignment
 Attempt random feasible reassignments of c
 if no valid move found **then continue**
 end if
 Compute cost difference Δ
 if $\Delta < 0$ or $\text{random}() < e^{-\Delta/T}$ **then**
 Accept new assignment
 if cost improves best **then**
 update S^*
 end if
 else
 Revert assignment
 end if
 end for
 $T \leftarrow \alpha \cdot T$
end while
return $\text{LOCALSEARCH}(S^*)$

3.1.5 Hyperparameters

The performance of the Simulated Annealing (SA) algorithm is influenced by several hyperparameters that control the cooling schedule and convergence behavior. The key parameters used in our implementation are:

- **Initial Temperature** T determines the initial probability of accepting worse solutions.
- **Minimum Temperature** $T_{\min}$ the stopping criterion for the cooling process.
- **Cooling Rate** α controls how quickly the temperature decreases.

These hyperparameters were manually tuned through empirical testing to ensure the algorithm returns high quality solutions within reasonable runtime. Different settings may lead to better results for specific tests, but the chosen values provide a good balance between exploration and convergence in our experiments.

Hyperparameters	Tested values	Optimal value
T	10, 20, 50, 100	100
$T_{\min}$	10^{-3} , 10^{-4} , 10^{-5} , 10^{-6}	10^{-6}
α	0.9999, 0.99999, 0.999999	0.999999

Table 1: Hyperparameters examined for SA Algorithm

3.2 Beam Search

3.2.1 Overview

Beam Search (BS) is a breadth-limited heuristic that maintains a fixed-size set (“beam”) of the best partial schedules at each step. At each iteration, all extensions of the current beam are generated, then pruned to keep only the top- W candidates by cost, thus balancing exploration and efficiency.

3.2.2 Strategy

Initial Beam

We start with a single empty state in the beam, where no classes have been assigned.

Beam Width

Maintain at most *beam_width* states at each level. After expanding all successors, sort by cost and truncate to keep only the best *beam_width* states.

Permutations

Optionally repeat the entire beam search from different random class-orderings to diversify search and select the best overall result.

3.2.3 Process

The beam-search process starts by considering classes one by one in a predefined order. At each step, we keep a small set of partial schedules (called the “beam”). For the next class, we take each partial schedule in the beam and try to place that class into every feasible combination of time slot and room that does not increase the number of days used by more than one. Each valid placement produces a new partial schedule, whose cost is updated accordingly. After generating all these new partial schedules, we rank them by their cost and retain only the best ones to form the beam for the next iteration.

Once all classes have been placed, we apply a simple local improvement: for each class, we check if moving it to an earlier slot would reduce the total days used. If such a move strictly lowers the timetable’s span, we perform it. We repeat this check for every class until no further reduction in days is possible.

3.2.4 Pseudo-code

The following pseudo-codes describe the implementation of the Beam Search algorithms used to solve the ETP. The pseudo-codes outline the steps involved in the scheduling process using these algorithms.

Algorithm 3 The Beam Search algorithm

Input: Class ordering π , beam width W , initial beam $B = \{\text{empty state}\}$

Output: Best solution found

for each class index i in π **do**

$B' \leftarrow \emptyset$

for each state $s \in B$ **do**

for each slot $1 \leq t \leq s.\text{cost} + 1$ **do**

for each feasible room r for class i **do**

if placement (i, t, r) valid in s **then**

$s' \leftarrow s$ with $(i \rightarrow (t, r))$, update $s'.$ cost

$B'.\text{add}(s')$

end if

end for

end for

end for

 Sort B' by increasing cost; keep top W

$B \leftarrow B'$

end for

return best state in B

3.3 Harmony Search

3.3.1 Overview

Harmony Search (HS) is a music-inspired meta-heuristic that iteratively improvises new “harmonies” (solutions) by combining existing good harmonies, adjusting pitches, and introducing randomness. It maintains a Harmony Memory (HM) of the best solutions and updates it over many improvisation iterations.

3.3.2 Strategy

Harmony Memory Initialization

Generate an initial HM of size hs_size by constructing feasible timetables via a greedy heuristic over random class orderings.

Improvisation

Create new harmony, with probability $hmcr$, copying each class’s slot from a randomly chosen harmony in HM . If a slot is copied, there is a probability par of adjusting it. Once all slots are determined, compute its cost and replace the worst harmony if it’s better.

3.3.3 Process

We build the Harmony Memory (HM) by generating hs_size complete timetables. For each harmony, we randomly shuffle the list of classes and assign them one by one to the earliest available slot and room that meet capacity and conflict requirements. These initial solutions provide a diverse starting set.

During each improvisation step, we create a new candidate by iterating over all classes in a fixed order. For each class, we decide (based on $hmcr$) whether to copy its slot from one of the existing harmonies (selected at random) or to choose a slot uniformly at random. If we copy a slot, we then apply a pitch adjustment with probability par , shifting the slot by a value drawn from $\{-bw, \dots, +bw\}$ and clipping it to stay within valid day indices. Once slots for every class are determined, we assign rooms greedily, placing each class into the first room that satisfies capacity and conflict constraints; if any class cannot be placed, the entire candidate is discarded. We evaluate the cost of a valid candidate and compare it to the worst harmony in HM . If the new cost is lower, we replace the worst harmony. This cycle of improvisation continues for a predetermined number of iterations, after which the best harmony remaining in HM is returned as the final timetable.

After scheduling every class, a local refinement is performed: each class is examined to see if relocating it to an earlier time slot decreases the total number of days used. If relocating a class results in a strictly shorter span, the move is made.

3.3.4 Pseudo-code

The pseudo-codes presented below detail the implementation strategies of the Harmony Search algorithms employed to solve the ETP, outlining the sequential steps involved in the scheduling procedure.

Algorithm 4 The Harmony Search algorithm

```
1: Input: Class set,  $hs\_size$ ,  $hmcr$ ,  $par$ ,  $bw$ ,  $hs\_iter$ 
2: Output: Best solution found
3: Initialize  $HM$  with  $hs\_size$  greedy solutions
4:  $best \leftarrow$  best harmony in  $HM$ 
5: for  $t = 1$  to  $hs\_iter$  do
6:   for each class  $i = 1$  to  $N$  do
7:     if  $random() < hmcr$  then
8:       pick slot  $s$  from a random harmony in  $HM$ 
9:       if  $random() < par$  then
10:         $s \leftarrow s + round(uniform(-bw, +bw))$ 
11:        clip  $s$  into  $[1, best.cost]$ 
12:       end if
13:     else
14:       pick random slot  $s \in [1, best.cost]$ 
15:     end if
16:      $slot\_imp[i] \leftarrow s$ 
17:   end for
18:   replace worst harmony in  $HM$  with new if  $c < worst.cost$ 
19:   if  $c < best.cost$  then
20:      $best \leftarrow$  new harmony
21:   end if
22: end for
23: return LOCALSEARCH( $best$ )
```

3.3.5 Hyperparameters

The performance of the Harmony Search (HS) algorithm is significantly influenced by several key parameters, each of which plays a crucial role in guiding the search process and determining the quality of the final solution. The main parameters considered in our implementation are as follows:

- **Harmony Memory Size** hs_size number of candidate solutions stored in memory.
- **Harmony Memory Consideration Rate** $hmcr$ probability of choosing a value from existing harmony memory.
- **Pitch Adjustment Rate** par probability of fine-tuning a chosen value.

- **Bandwidth** bw maximum amount of adjustment applied during pitch adjustment.
- **HS Iterations** hs_iter total number of improvisation steps.

In our experiments, each parameter was varied within a predefined range and selected manually to achieve the best trade-off between solution quality and computational effort.

Hyperparameter	Tested Values	Optimal Value
hs_size	10, 15, 20, 25, 30	25
$hmcr$	0.80, 0.85, 0.90, 0.95	0.95
par	0.2, 0.3, 0.4, 0.5, 0.6	0.4
bw	1, 2, 5	1
hs_iter	10^3 , 10^4 , 10^5 , 10^6	10^6

Table 2: Hyperparameters examined for the Harmony Search algorithm

4 Result and Analysis

4.1 Result

4.1.1 CP model and MIP model solver

Exact Solution

Input sizes		CP		MIP	
N	M	f	t(ms)	f	t(ms)
5	2	5	7	5	35
6	2	3	12	3	85
7	3	3	18	3	162
8	3	3	17	3	278
9	4	4	27	4	123
10	3	4	26	4	164
11	4	6	31	6	423
12	4	4	43	4	577
13	4	5	142	5	902
14	4	8	93	8	1334
15	4	9	72	9	28636
16	4	8	173	8	21724
17	4	10	188	F	F
18	5	7	189	F	F
19	5	8	13841	F	F
50	10	5	7345	F	F

Table 3: Results only exact solution of CP solver and MIP solver

All Solution

As a result of the large size and intricate nature of the optimization problems under consideration, we were only able to obtain feasible solutions for test instances with input size smaller than or equal to (50×10) for the CP solver, and (16×4) for the MIP solver, within a time limit of 300 seconds. This indicates that the optimization problems are highly complex and computationally demanding, requiring significant computational resources to solve. To ensure the reliability of our findings, we conducted ten runs of each solver and computed the average results, thereby mitigating the potential impact of random variations and ensuring the robustness of our numerical experiments.

Input sizes		CP				MIP			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	7	5	5	5	35
6	2	3	3	3	12	3	3	3	85
7	3	3	3	3	18	3	3	3	162
8	3	3	3	3	17	3	3	3	278
9	4	4	4	4	27	4	4	4	123
10	3	4	4	4	26	4	4	4	164
11	4	6	6	6	31	6	6	6	423
12	4	4	4	4	43	4	4	4	577
13	4	5	5	5	142	5	5	5	902
14	4	8	8	8	93	8	8	8	1334
15	4	9	9	9	72	9	9	9	28636
16	4	8	8	8	173	8	8	8	21724
17	4	10	10	10	188	10	10	10	301973
18	5	7	7	7	189	7	8	7.67	303064
19	5	8	8	8	13841	8	8	8	302770
50	10	5	5	5	7345	5	5	5	321161
51	10	7	8	7.33	300154	9	9	9	325693
52	11	8	9	8.33	300164	15	16	15.33	325189
53	11	14	14	14	300537	15	15	15	315767
54	11	9	11	10	300218	9	11	10.33	316766
55	11	14	14	14	300202	15	15	15	314033
56	12	9	10	9.67	300191	13	15	14	326837
57	12	9	12	11	300184	18	18	18	337534
58	12	9	11	10.33	300222	23	27	25.33	339576
59	13	9	9	9	300262	17	17	17	342626
100	10	18	18	18	300696	F	F	F	F
200	10	184	210	195.33	302356	F	F	F	F
300	10	F	F	F	F	F	F	F	F
400	10	F	F	F	F	F	F	F	F
500	10	F	F	F	F	F	F	F	F
600	11	F	F	F	F	F	F	F	F
700	11	F	F	F	F	F	F	F	F
800	11	F	F	F	F	F	F	F	F
900	10	F	F	F	F	F	F	F	F
1000	10	F	F	F	F	F	F	F	F
1000	20	F	F	F	F	F	F	F	F
2000	20	F	F	F	F	F	F	F	F
3000	20	F	F	F	F	F	F	F	F

Table 4: Average results of CP solver and MIP solver

4.1.2 Heuristic solver

Input sizes		SA				HS				BS			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	209	5	5	5	527	5	5	5	6
6	2	4	4	4	204	3	3	3	792	3	3	3	25
7	3	4	4	4	219	4	4	4	901	3	3	3	40
8	3	3	4	3.4	234	3	3	3	1068	3	3	3	61
9	4	5	5	5	254	4	4	4	1043	4	4	4	47
10	3	4	4	4	272	4	4	4	984	4	4	4	58
11	4	6	7	6.8	291	6	7	6.2	1244	6	6	6	69
12	4	5	6	5.4	293	4	5	4.2	1189	4	4	4	100
13	4	7	8	7.6	308	6	7	6.3	1308	5	5	5	84
14	4	8	9	8.6	328	8	8	8	1133	8	8	8	46
15	4	10	11	10.6	337	10	11	10.3	1222	9	9	9	65
16	4	9	9	9	336	8	9	8.4	1449	8	8	8	82
17	4	11	11	11	342	10	11	10.4	1390	10	10	10	118
18	5	9	9	9	349	8	9	8.4	1507	7	7	7	169
19	5	9	9	9	363	9	11	9.6	1690	8	8	8	156
50	10	11	11	11	486	9	12	10.9	3371	8	9	8.7	1214
51	10	11	11	11	486	9	11	10.3	3441	8	9	8.6	1459
52	11	11	11	11	487	10	11	10.8	3888	9	10	9.4	1608
53	11	15	15	15	507	14	16	14.6	3432	14	14	14	1202
54	11	12	13	12.5	510	12	14	12.9	3976	11	11	11	849
55	11	16	17	16.4	551	16	18	16.7	4170	14	15	14.9	647
56	12	12	12	12	488	11	13	12.2	3900	10	11	10.6	1353
57	12	12	12	12	492	12	13	12.4	4377	11	11	11	1910
58	12	12	13	12.3	511	11	13	12.3	4479	11	11	11	1151
59	13	12	12	12	510	11	14	12.3	4174	10	11	10.8	2091
100	10	20	21	20.5	603	21	23	22.2	5511	20	22	20.9	1892
200	10	35	35	35	745	34	36	34.9	10679	34	35	34.3	5330
300	10	47	53	50	1097	50	54	52.2	15854	50	51	50.2	11402
400	10	63	64	63.4	1818	65	69	67.8	21066	64	66	65.1	19995
500	10	69	74	71	2002	71	76	73.9	26951	72	75	73.7	36479
600	11	90	95	92.6	3306	92	96	93.7	30783	91	94	92.8	48787
700	11	89	93	91	3336	94	97	94.9	34217	92	94	93.2	103180
800	11	158	160	159.1	17706	160	162	161	40177	160	162	161.4	97782
900	10	125	127	126.3	10848	130	133	131.6	43805	128	131	129.9	138246
1000	10	184	186	185	25959	190	195	191.8	48348	185	188	186.7	171272
1000	20	120	123	121.5	7433	124	129	125.8	49775	122	125	124	330373
2000	20	174	176	175	38520	178	183	180.8	200900	F	F	F	F
3000	20	178	179	178.4	156880	180	182	181	151614	F	F	F	F

Table 5: Results of Heuristics solver

4.2 Analysis

4.2.1 Exact Solution

Our evaluation of the accuracy of each algorithm revealed that CP and MIP were capable of generating exact solutions for a subset of test cases:

- CP was able to produce exact solutions for tests with sizes:
(5×2), (6×2), (7×3), (8×3), (9×4), (10×3), (11×4), (12×4), (13×4), (14×4), (15×4), (16×4), (17×4), (18×5), (19×5), (50×10), (51×10), (52×11), (53×11)
- MIP was only able to produce exact solutions for tests with sizes:
(5×2), (6×2), (7×3), (8×3), (9×4), (10×3), (11×4), (12×4), (13×4), (14×4), (15×4), (16×4)

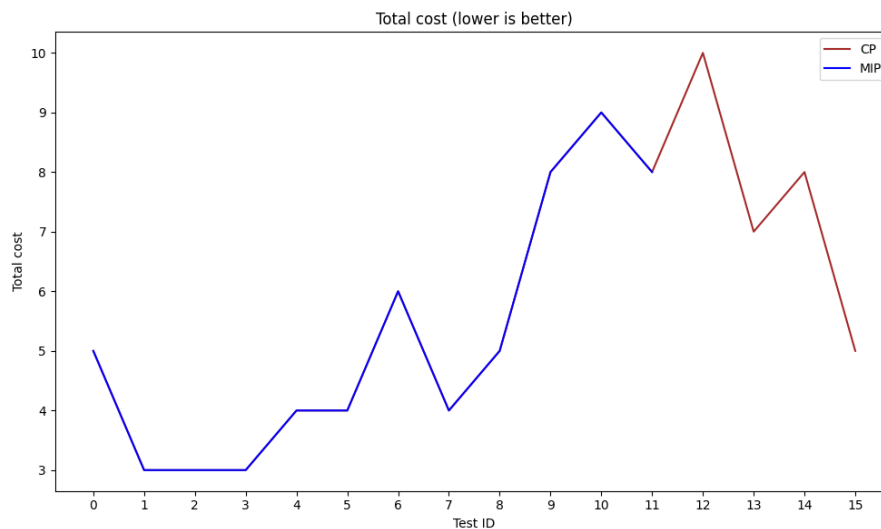


Figure 1: Results compare only exact solution

4.2.2 All Solution

Our analysis of the solutions generated by five distinct algorithms revealed the following:

- CP is unable to handle datasets larger than (200×10).
- MIP is unable to handle datasets larger than (59×13).
- BS fails to solve two test cases: (2000×20) and (3000×20).
- Both SA and HS are capable of handling all test cases.

In terms of total cost, our findings indicate:

- MIP performs the worst.
- CP and MIP generate equivalent results on small test cases.
- SA, HS, and BS demonstrate very similar trends and clearly outperform CP and MIP in terms of total cost for test IDs ≥ 25 (i.e., data sizes larger than (200×10)).

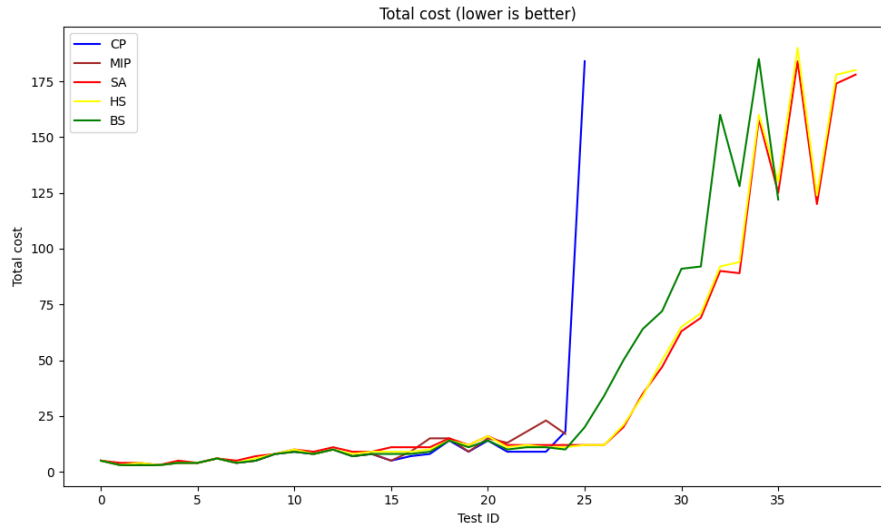


Figure 2: Results compare all solutions

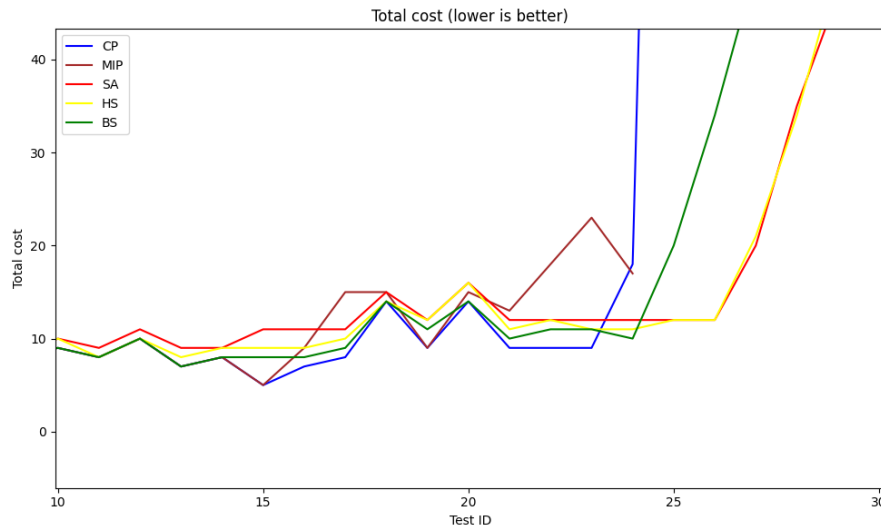


Figure 3: Results compare all solutions from test 10 to 30

4.2.3 Runtime

The running time of each algorithm was also evaluated, and our findings indicate that:

- MIP reaches the time limit of 300 seconds for all tests with size greater than or equal (17×4).
- CP reaches the time limit of 300 seconds for all tests with size greater than or equal (51×10).
- BS reaches the time limit of 300 seconds for all tests with size greater than or equal (1000×20).

- SA and HS have short runtimes, with most tests completing in under 50 seconds, and the two largest test cases (2000×20) and (3000×20) taking at most 200 seconds.

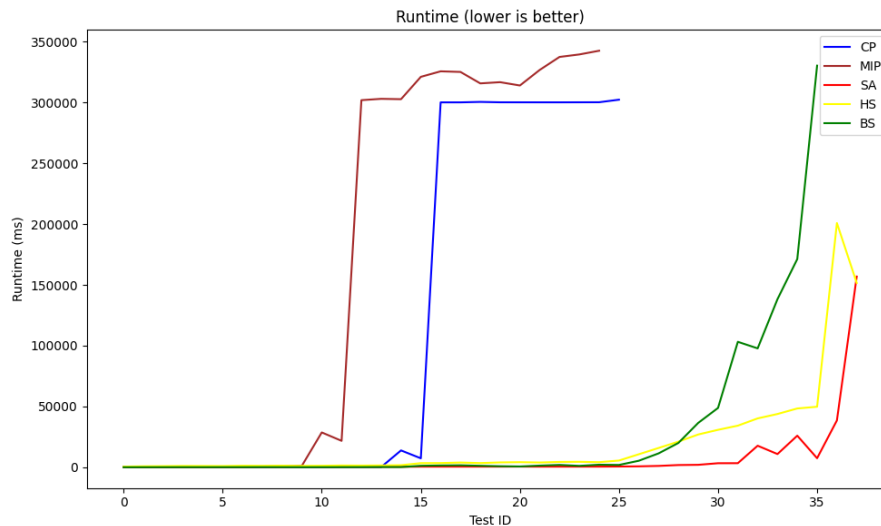


Figure 4: Run time of all 35 tests

5 Conclusion

In conclusion, our evaluation demonstrates that the heuristic algorithms are both accurate and efficient, with the SA algorithm standing out for its combination of fast runtimes and strong solution quality. These characteristics make heuristic methods particularly well-suited for practical applications that demand both speed and accuracy. While the CP solver remains a viable option, especially for smaller datasets, the MIP solver consistently underperforms and is therefore not recommended.

6 Limitations and Future Research

Our analysis also revealed several limitations and potential directions for future research. Firstly, the study was conducted using a fixed set of synthetic test cases, and did not incorporate real-world data. Future work could focus on evaluating the algorithms using real-life datasets from universities to better assess their practical effectiveness.

In addition, further research could explore ways to enhance the performance of the heuristic algorithms while preserving their accuracy and efficiency. Another promising direction is the development of hybrid approaches that combine elements of constraint programming (CP), mixed-integer programming (MIP), and heuristic methods to achieve improved results.

Moreover, this study primarily addressed the theoretical performance of the algorithms

and did not investigate their implementation in real-world operational environments. Future studies could examine the feasibility, scalability, and robustness of these algorithms when deployed in practical scheduling systems.

In summary, this work provides valuable insights into the comparative performance of various algorithms for the considered optimization problem, highlights their respective strengths and limitations, and outlines key areas for continued research.

7 Visualization

For visualization purposes, we generated figures to illustrate the results of the CP solver and the three heuristic algorithms. Due to the MIP solver's significantly longer runtimes and consistently poor performance, we excluded it from the visual analysis. As such, the figures focus solely on the comparative performance of the CP and heuristic solvers.

The following figures present a comparison between the CP solver and the heuristic approaches:

```
10 3
72 77 71 71 53 45 53 53 66 70
79 53 70
16
1 2
1 3
1 8
1 10
2 5
2 9
3 6
3 9
4 10
5 8
5 10
7 8
7 9
7 10
8 9
9 10
```

Slot 1
 Slot 2
 Slot 3
 Slot 4

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

(a) CP solver

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

(b) SA solver

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

(c) BS solver

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

(d) HS solver

Figure 5: Result for test size (10×3)

References

- [1] E. S. K. Siew, S. N. Sze, S. L. Goh, G. Kendall, N. R. Sabar and S. Abdullah, *A Survey of Solution Methodologies for Exam Timetabling Problems*, IEEE Access, 2024.
- [2] Ruggero Bellio, Sara Ceschia, Luca Di Gaspero, Andrea Schaerf, Tommaso Urli, *Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem*, Elsevier, 2016.
- [3] Soppari, Kavitha and Rankhambe, Jayashree, *Optimization of Examination Timetable Using Harmony Search Hyper-Heuristics (HSHH)*, International Journal of Computer Science and Information Technology, 2014.